This lecture introduces the concept of functions in programming, explaining their purpose, benefits, and implementation in Python. The main points covered are:

- Introduction to Functions:
    - Functions address the reusability problem in scripting-style coding.
    - They allow you to encapsulate a block of code that can be called multiple times, avoiding redundant code.
    - Functions promote modularity, making code easier to maintain and connect.
- Benefits of Using Functions:
    - Increased code reusability.
    - Improved code modularity and maintainability.
    - Avoidance of repetitive coding.
- Built-in Functions:
    - Examples like `len()` and `type()` are used to illustrate how pre-built functions work.
    - These functions encapsulate complex logic that users can access without needing to know the implementation details.
- Defining and Calling Functions:
    - Functions are defined using the `def` keyword in Python.
    - The syntax includes the function name, parentheses for arguments, and a colon to begin the function body.
    - Functions are called by using their name followed by parentheses.
- Return Statements:
    - The `return` keyword is used to return a value from a function.
    - Unlike `print`, which returns `None` and prevents further operations on the output, `return` allows you to use the function's output in other operations.
    - Functions can return multiple values, which are packed into a tuple.
- Function Arguments:
    - Functions can accept input arguments, allowing them to operate on different data.
    - When defining a function, you specify the arguments it expects.
    - When calling a function, you must provide the required arguments.
- Practical Examples:
    - Examples of functions for addition, multiplication, checking even numbers, and calculating powers are provided.
    - A detailed example of converting a code snippet for filtering even numbers from a list into a reusable function is shown.
- Docstrings:
    - Docstrings are used to document functions, providing a description of what the function does, its arguments, and its return value.
    - They are defined using triple quotes (`"""`) at the beginning of the function body.
    - Docstrings are displayed when you hover over the function name in an IDE or use the `help()` function.
- Argument and Return Type Hints:
    - Type hints can be added to function arguments and return values to indicate the expected data types.

        - These hints are not enforced by Python but serve as documentation and can be used by static analysis tools.
        - Example: `def even_list_filter(L: list) -> list:`
- Key Takeaways:
    - Functions are essential for writing reusable, modular, and maintainable code.
    - The `return` statement is crucial for using function outputs in other operations.
    - Docstrings and type hints are important for documenting functions and educating users.

In summary, the lecture provides a comprehensive introduction to functions in Python, covering their definition, usage, documentation, and best practices for writing effective and reusable code.

-